



Enhanced Differential Cache Attacks on SM4 with Algebraic Analysis and Error-Tolerance

Xiaoxuan Lou^{1,2}, Fan Zhang^{2,3,4(✉)}, Guorui Xu³, Ziyuan Liang^{1,3,4},
Xinjie Zhao⁵, Shize Guo⁵, and Kui Ren^{3,4}

¹ Department of Electronic Engineering,
College of Information Science and Electronic Engineering,
Zhejiang University, Hangzhou 310027, China

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing 100878, China

³ School of Cyber Science and Technology,
College of Computer Science and Technology, Zhejiang University,
Hangzhou 310027, China
fanzhang@zju.edu.cn

⁴ Alibaba-Zhejiang University Joint Institute of Frontier Technologies,
Zhejiang University, Hangzhou 310027, China

⁵ The Institute of North Electronic Equipment, Beijing 100878, China

Abstract. Block ciphers with Feistel structures are vulnerable to a specific type of cache attacks named differential cache attacks. The attacks leverage side-channel leakages from cache and differential property of cipher component to reveal the master key of cipher. In this paper, we combine the algebraic analysis to enhance the attacks, and propose a novel method named *Algebraic Differential Cache Attack (ADCA)*. By converting both cipher and cache leakages to algebraic equations, ADCA can reveal the cipher key automatically with the help of the SAT solver, which allows the analysis on much deeper rounds and makes a considerable reduction in attack complexity. When it is applied to the block cipher SM4, 10 plaintexts are enough to reveal the master key in 8-rounds analysis, while the traditional differential cache attack needs 20 ones. Finally, to eliminate the impact from noise, an error-tolerant method is proposed to deduce cache events from the leakage traces. It vastly enhances the robustness of attack, and makes the attack more practical. The experimental results show that the error-tolerant ADCA can correctly reveal the master key even when the uncertainty rate of cache events reaches to 60%.

Keywords: SM4 · Side channel attacks · Differential cache attacks · Algebraic analysis · Error-tolerance

1 Introduction

Cache attacks are a class of side-channel attacks that monitor the leakages from cache to extract the cipher key. [15] first proposed such attacks theoretically, and

discussed the feasibility of revealing the master key of DES with captured cache leakages. Then [20] followed the idea and demonstrated the first practical cache attack. After that, a number of cache attacks involving different techniques have been presented. Depending on the side-channel leakages used, cache attacks can be divided into three classes: time-driven [3, 20], access-driven [12, 14] and trace-driven [5, 17]. These attacks pose a significant threat to various ciphers, including AES, CLEFIA, Camellia, and the target of this paper, SM4. As the standard commercial block cipher in China, SM4 [2] (formerly known as SMS4) has a typical unbalanced Feistel structure. In recent years, with the wide application of SM4, multiple general cryptography libraries start to support the cipher, like the latest OpenSSL 1.1.1 [1]. Since it was released in 2006, SM4 has been subjected to extensive researches. Numerous works were arisen to evaluate the reliability and security of the cipher, which roughly contain two primary categories: cache attacks and algebraic analysis.

Cache attacks are usually the first choices to analyze the block cipher like SM4. [4] carried out an effective cache attack on SM4, showing that 5000 power traces are enough to recover the master key. [22] used access-driven cache attack to analyze the first four rounds of encryption in the SM4, and revealed the cipher key within 80 plaintext samples. Among the cache attacks on SM4, there is a specific class named *Differential Cache Attack (DCA)*. In [17], trace-driven attacks were first combined with differential analysis, where the differential properties of S-Boxes are used to recover the key of CLEFIA in less than 2^{14} traces. This novel attack is the so-called *Differential Cache Attack*, which is further improved in [18] and also applied to Camellia in [16]. DCA is very suited to ciphers with Feistel structures, like the SM4. [13] proposed a differential cache attack on SM4, it used 2^{10} plaintexts to recover the master key of SM4 from the power traces for the 64-byte cache line.

Besides the cache attacks, algebraic analysis is also an available way to appraise the cipher. Some previous works targeted on the algorithm structure of SM4 to reveal partial cipher key, like 14-round square attack [23], 17-round impossible differential analysis [6], 22-round differential analysis [21] and 23-round linear analysis [7]. Beyond that, [10] described SM4 cipher as systems of quadratic equations over $GF(2)$ and $GF(2^8)$, and also estimated the complexities of XL algorithm for solving the two equations systems. And [8] performed algebraic analysis on 5-round SM4 leveraging the Gröbner basis and Minisat solver. With the algebraic analysis, the attacker can effectively decrease the search space of master key, and finally reveal it with simple brute force.

In this paper, we propose an enhanced differential cache attack on SM4, which combines with algebraic analysis and adopts an error-tolerant method to deduce cache events. It releases the analyst from sophisticated analysis and improves the robustness of attack. To thoroughly evaluate the resistance of SM4 against such attack, we first present a traditional DCA method to reveal the master key of SM4. Experiments show that it requires 20 plaintexts when both cache hits and cache misses are analyzed. However, such traditional method is sophisticated and limited by the depth of analysis rounds. To enhance the

attack, we combine it with algebraic analysis, proposing a novel attack named *Algebraic Differential Cache Attack (ADCA)*. Such attack derives the algebraic equations of both cipher algorithm and cache leakages, then sets them up as the system of equations, which is further solved by the SAT solver automatically. The outputted solution would contain the master key so that the attacker can extract it. Since the new attack only requires the transformation of algebraic equations, and leaves the complicated analysis to the SAT solver, it is much simpler and more generic than previous DCA methods. Based on the results of experiments, ADCA can reveal the key of SM4 with only 10 plaintext by analyzing the first 8 rounds. Finally, in order to make the attack more practical, we introduce an error-tolerant method to deduce cache hit/miss events from the captured side-channel traces. With two stated thresholds, only the events whose leakage values locate in the range would be analyzed, and the others would be marked as uncertain and be discarded. It effectively prevents the impact of noise and improves the robustness of attacks. The enhanced ADCA can reveal the key when the uncertainty rate of deduced cache events is less than 60%.

The contributions of the paper are as follows:

- We give a security analysis of SM4 against *Differential Cache Attack (DCA)*, and present a practical DCA method that uses 20 plaintexts to reveal the master key of SM4.
- We combine algebraic analysis with DCA to give a novel attack named *Algebraic Differential Cache Attack (ADCA)*, illustrating the process of constructing equations of cipher algorithm and cache events.
- We demonstrate a practical ADCA on SM4, which extract the master key of SM4 with analyzing first 8 rounds and only 10 plaintexts. It greatly reduces the attack complexity and bypasses the sophisticated analysis.
- We propose an error-tolerant enhancement to improve the robustness of ADCA, eliminating the influence from uncertain cache events caused by noise.

The rest of this paper is organized as follows: Sect. 2 introduces the SM4 cipher and specifies the differential cache attack model. Section 3 demonstrates the details of proposed DCA and shows the results of attack experiments. Then in Sect. 4, the attack model of ADCA and relevant theoretical basis are introduced. Section 5 describes the detailed implementation process of such novel attack, while Sect. 6 evaluates its performance. Section 7 discusses the error-tolerant enhancement and Sect. 8 concludes the paper.

2 Preliminaries

2.1 Notation

For the sake of legible expression, notations used in this paper are listed first. Throughout the paper, MK denotes the master key of SM4, $(X_i, X_{i+1}, X_{i+2}, X_{i+3})$ are four intermediate variables inputted to the round i ($0 \leq i < 32$), and (Y_0, Y_1, Y_2, Y_3) denote the final ciphertexts. rk_i is the round

key generated from MK and used for i -th round. Let q_j denote the j -th S-Box lookup in the execution of SM4, λ denote the number of table lookups considered in the attack, $0 \leq j < \lambda$. The lookup index of q_j is denoted as l_j . Suppose the size of cache line is 2^μ bytes, and that of S-Box is 2^δ bytes. As a result, the S-Box occupies $2^{\delta-\mu}$ cache lines, and the higher $b = \delta - \mu$ bits of lookup index l_j identify which cache line contains the access target. Let $\langle l_j \rangle$ denote the most significant b bits leaked from the cache accesses in the attack.

2.2 SM4 Cipher

SM4 has a 32-rounds iterative Feistel structure, as shown in Fig. 1. The cipher adopts a 128-bits master key, which further generates 32-bits round keys for each round. Equation (1) depicts the encryption process of SM4:

$$\begin{aligned} X_{i+4} &= F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i) \\ &= X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i) \end{aligned} \quad (1)$$

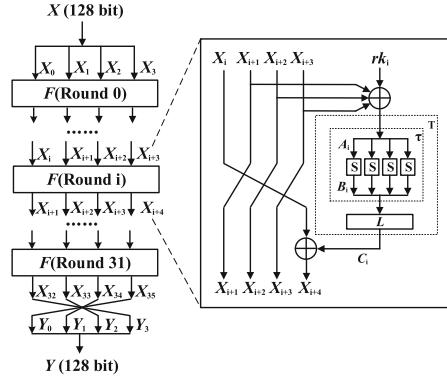


Fig. 1. The structure of SM4 cipher.

where T is a function composed of the nonlinear part τ and the linear part L , i.e., $T(.) = L(\tau(.))$. The nonlinear conversion τ is achieved with four table lookups on the 8×8 S-Box, whose inputs are $A_i = (a_{i,0}, a_{i,1}, a_{i,2}, a_{i,3})$ and outputs are $B_i = (b_{i,0}, b_{i,1}, b_{i,2}, b_{i,3})$. Each pair of $a_{i,t}$ and $b_{i,t}$ corresponds to t -th S-Box lookup in round i , $0 \leq t < 4$. For the following linear conversion L , it takes B_i as input and outputs $C_i = (c_{i,0}, c_{i,1}, c_{i,2}, c_{i,3})$ like shown in Eq. (2). As the result of function T , C_i further XORs with X_i to generate X_{i+4} .

$$C_i = L(B_i) = B_i \oplus (B_i \lll 2) \oplus (B_i \lll 10) \oplus (B_i \lll 18) \oplus (B_i \lll 24) \quad (2)$$

2.3 Differential Cache Attacks

Following the explanation of differential cache attacks in [17], we introduce how to perform such attack and propose the attack model, as shown in Fig. 2. Suppose the cache has been flushed before the encryption, and there are λ S-Box lookups being analyzed in the attack. For a certain lookup q_j among them, to figure out its index l_j , we let U_j denote a set of public variables (plaintexts) and V_j denote a set of secret variables (round keys), then introduce the function $f(\cdot)$ that computes the l_j from U_j and V_j . Recall that $\langle l_j \rangle$ ($0 \leq j < \lambda$) denotes the most significant b bits of l_j , which indicate the index of accessed cache line. Hence, the relationship between different $\langle l_j \rangle$ can be deduced from the cache hit/miss sequences, which also provides constrains to help the revealing of secret variable V_j .

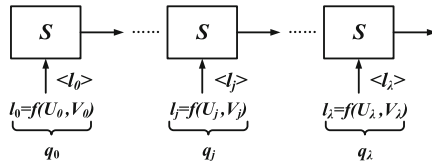


Fig. 2. Differential cache attack model.

Assume the cache has been flushed beforehand, so the first lookup q_0 would cause a cache miss. Then we take the second S-Box lookup q_1 for example to discuss. The memory access in the q_1 can be either cache hit or cache miss, the former indicates q_1 accesses the same cache line with q_0 , i.e., $\langle l_0 \rangle = \langle l_1 \rangle$, so that Eq. (3) can be derived. The latter means the target data is absent in the cache, so q_1 would load it and there is Eq. (4).

$$\langle l_0 \rangle = \langle l_1 \rangle \implies \langle f(U_0, V_0) \rangle_b = \langle f(U_1, V_1) \rangle_b \quad (3)$$

$$\langle l_0 \rangle \neq \langle l_1 \rangle \implies \langle f(U_0, V_0) \rangle_b \neq \langle f(U_1, V_1) \rangle_b \quad (4)$$

In this way, the cache access events are transformed to the equations about U_j and V_j . To recover the key, multiple plaintext samples are sent to the cipher, and then differential equations between them are constructed. For instance, assume the input plaintexts X and X' both cause a cache hit in q_1 , so they both meet Eq. (3), which further derives the differential equation as below:

$$\langle f(U_0, V_0) \oplus f(U'_0, V_0) \rangle_b = \langle f(U_1, V_1) \oplus f(U'_1, V_1) \rangle_b \quad (5)$$

Because of the feature of Feistel structure, V_1 in Eq. (5) generally can be eliminated, only leaving the V_0 unknown. And based on the property of the S-Box, the value of V_0 can be uniquely recovered with a certain number of plaintext samples. Hence, with the differential cache attack, the adversary can leverage a number of plaintext pairs to reduce the search space of secret variables and finally reveal the correct key.

3 Differential Cache Attack on SM4

In this section, we first present a traditional differential cache attack on the SM4. The attack is improved from [11] and analyzes the first four rounds of cipher to reveal the master key MK . The work lays a foundation for the design of subsequent enhanced attack method.

For the first round of SM4, it needs to perform four S-Box lookups whose inputs, namely lookup indexes, are denoted as $a_{0,t}$, ($0 \leq t < 4$). Each $a_{0,t}$ is computed as: $a_{0,t} = X_{1,t} \oplus X_{2,t} \oplus X_{3,t} \oplus rk_{0,t}$, where $X_{i,t}$ and $rk_{i,t}$ denote the t -th block (each block is 8-bit) of variables X_i and rk_i . As mentioned above, since q_0 always be a cache miss, the analysis starts from q_1 . If q_1 results in a cache hit, there should be $\langle l_0 \rangle = \langle l_1 \rangle$ (i.e., $\langle a_{0,0} \rangle = \langle a_{0,1} \rangle$), and Eq. (6) is derived. The equation offers the possible values of $\langle rk_{0,0} \oplus rk_{0,1} \rangle$.

$$\begin{aligned} \langle a_{0,0} \rangle = \langle a_{0,1} \rangle &\implies \langle rk_{0,0} \oplus rk_{0,1} \rangle = \\ \langle X_{1,0} \oplus X_{2,0} \oplus X_{3,0} \oplus X_{1,1} \oplus X_{2,1} \oplus X_{3,1} \rangle \end{aligned} \quad (6)$$

On the contrary, if q_1 causes a cache miss, the equation would give the impossible values of $\langle rk_{0,0} \oplus rk_{0,1} \rangle$. Similarly, for the following lookups, depending on their cache accessing state, the attacker can extract possible values and discard impossible values of corresponding round keys. As a result, after the analysis of the first round, the values of $\langle rk_{0,0} \oplus rk_{0,1} \rangle$, $\langle rk_{0,0} \oplus rk_{0,2} \rangle$ and $\langle rk_{0,0} \oplus rk_{0,3} \rangle$ can be recovered. With the same way to analyze deeper rounds, finally the first three round keys rk_0 , rk_1 , rk_2 and partial bits of rk_3 can be recovered. The remaining unknown bits, lower $(8 - b)$ bits in each $rk_{3,t}$, can be revealed with brute force, whose search space is 2^{32-4b} . Finally the master key can be recovered from these revealed round keys based on key scheduling.

The attack method has been applied to a SM4 cipher implemented on the 32-bit ARM microprocessor NXP LPC2124, whose cache line has 16 bytes [9]. The S-Box used in the SM4 cipher is of size 256 bytes. Hence, the number of bits for indexing the cache lines is $b = 4$. The electromagnetic radiation of microprocessor is captured by the EMK near-field probe on the oscilloscope, which is further used to depict the leakage traces. To evaluate the efficiency of the attack, we focus on the relation between the search space of round keys and the number of plaintext samples, depicting the variation tendency of such relation. For the first round, as shown in Fig. 3(a), the increasing analyzed plaintexts cause the search space of rk_0 to decrease rapidly. By analyzing both cache hits and cache misses, we find that 30 plaintexts are enough to reduce the search space from 2^{32} to 2^{20} , while only considering cache miss needs 35 plaintexts. The second round has analogous variation trend with the first round, like illustrated in Fig. 3(b). The search space of round keys (rk_0 and rk_1) decreases from 2^{64} to 2^{16} with the increasing of analyzed plaintexts. As for the following rounds, e.g., the third round and the fourth round, the relation between such two factors are pretty much similar, guaranteeing the reduction of round keys' search space with the increase of analyzed plaintexts.

After the analysis of the first four rounds, we have revealed whole rk_0 , rk_1 , rk_2 and partial rk_3 , so that the search space of them is reduced from 2^{128} to 2^{16} ,

as depicted in Fig. 3(c). It requires 20 plaintext samples when both cache hit and miss are analyzed, remaining 16 bits in the rk_3 unknown, which are easy to be solved with the brute force. Finally, according to the key schedule algorithm of SM4, the master key is calculated from the revealed (rk_0, rk_1, rk_2, rk_3) .

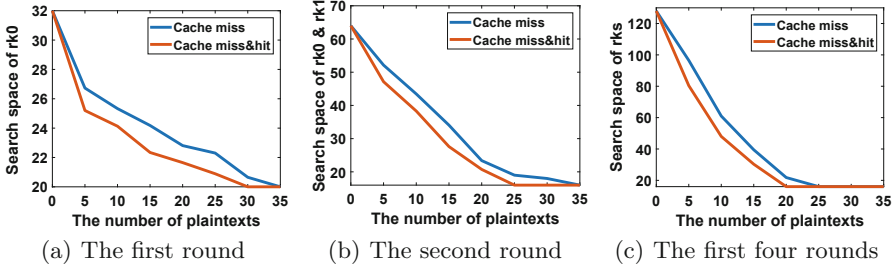


Fig. 3. Relation between the search space of round keys and the number of plaintexts.

4 Algebraic Differential Cache Attack

4.1 The Defects of DCA

Even above *Differential Cache Attack* method can recover the master key of SM4 effectively, there are still some exposed defects, making it less practical in the real scenarios. Firstly, the analysis is so complicated that generally requires analyst to own sophisticated knowledge, and the complexity would grow exponentially with the increase of analysis rounds, the overlarge search space remained from previous rounds would also cause the current round too complex to analyze. Secondly, the traditional DCA follows the *divide and conquer* strategy to recover the round keys block by block, which does not fully utilize the correlation between the algorithm structure and the round keys.

4.2 Attack Model

In this section we enhance the differential cache attack with the algebraic analysis and propose a novel attack named *Algebraic Differential Cache Attack* (ADCA). The attack model of ADCA is shown in Fig. 4, which contains three phases: (S1) Construct algebraic equations of the cipher; (S2) Deduce cache hit/miss sequences from the captured leakage traces then construct algebraic equations for these cache events; (S3) Use the SAT solver, which is commonly adopted in algebraic analysis, to solve the system of equations consisting of equations derived from (S1) and (S2), finally recover the master key. We will discuss these three phases in detail below.

Construct Algebraic Equations of Cipher. In essence, revealing the cipher key is equivalent to finding the solution that satisfies the algebraic equations of

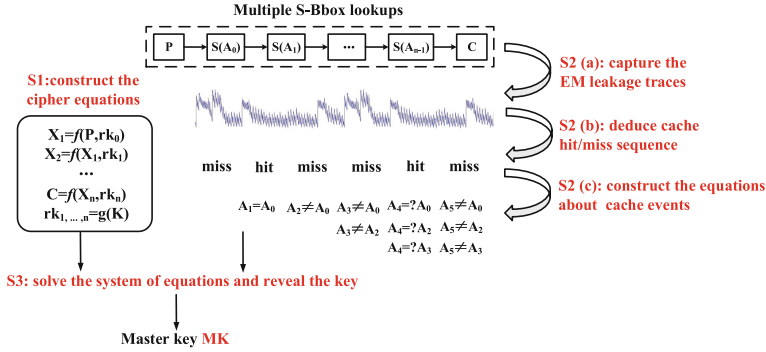


Fig. 4. The attack model of ADCA.

cipher. Hence, we need to first construct the algebraic equations of the cipher, which describe the algorithm with the plaintext X , the ciphertext Y , the round key rk and the master key MK . The detailed process of constructing equations of SM4 will be shown in Sect. 5.1.

Construct Algebraic Equations of Cache Events. However, only the cipher equations are not enough to reveal the master key. It is necessary to introduce additional information, like cache access sequence, to provide more constraints for analysis. Hence, the cache events deduced from captured leakage traces are transformed to algebraic equations, which can be seen as the kernel of ADCA. Such phase includes three steps, i.e., S2(a) to S2(c) in Fig. 4. First, multiple SM4 encryptions with different plaintexts are carried out, and at the same time, the electromagnetic radiation during each encryption process is also monitored. Then the cache events are deduced based on the captured traces, which are further used to construct the algebraic equations. The detailed process of constructing cache event equations is shown below.

Follow the notations defined in the Sect. 2.1, for the j -th lookup q_j , the most significant b bits of its index l_j , i.e., $\langle l_j \rangle$, can be revealed from the cache access, as long as q_j hits a certain cache line. Assume there have been r cache misses before the analyzed lookup q_j , and the set of lookup indexes of these cache misses is denoted as $S_M = \{l_{m1}, l_{m2}, \dots, l_{mr}\}$. If q_j results in a cache hit, it means the lookup accesses one of the cache lines that have been loaded before, so that the index l_j must have the same most significant b bits with one of elements in the S_M . Hence, similar with the analysis in the traditional DCA, we can get the set of possible values of $\langle l_j \rangle$, denoting as set $L_j = \{\langle l_{m1} \rangle, \langle l_{m2} \rangle, \dots, \langle l_{mr} \rangle\}$.

In order to improve the efficiency of comparing the values of $\langle l_j \rangle$ and $\langle l_{mt} \rangle$, one element in the L_j ($1 \leq t \leq r$), each $\langle l_{mt} \rangle$ is substituted with a b -bits variable, which is denoted as $D_t = \langle l_{mt} \rangle = d_t^1 d_t^2 \dots d_t^b$. Besides that, b single-bit variables denoted as e_t^k ($1 \leq k \leq b$), and one single-bit variable c_t are also introduced,

whose values are:

$$e_t^k = \neg(l_j^k \oplus d_t^k), \quad c_t = \prod_{k=1}^b e_t^k \quad (7)$$

where e_t^k shows if the k -th bit in the $\langle l_j \rangle$ and $\langle l_{mt} \rangle$ are equal, and c_t indicates whether all b bits of the $\langle l_j \rangle$ are equal to that of $\langle l_{mt} \rangle$. If $l_j^k = d_t^k$, there is $e_t^k = 1$; otherwise $e_t^k = 0$. As for the c_t , it has $c_t = 1$ if $\langle l_j \rangle = \langle l_{mt} \rangle$, or else $c_t = 0$.

As a result, if q_j causes a cache hit, there must be one and only one $\langle l_{mt} \rangle = \langle l_j \rangle$ in the set L_j , which is equivalent to only having one $c_t = 1$ in the set $C = \{c_1, c_2, \dots, c_r\}$. Such feature can be used to construct algebraic equations as below:

$$\prod_{c_t \in C_2} c_t = 0, \quad \prod_{c_t \in C} (c_t \oplus 1) = 0 \quad (8)$$

where C_2 is a two-element subset of set C . Then, we introduce two important parameters, the number of introduced variables (V) and the number of constructed equations (EQ). In this case, their values can be computed as:

$$V = (b+1) \times r, \quad EQ = 1 + (b+1) \times r + \frac{r \times (r-1)}{2} \quad (9)$$

where V includes e_t^k and c_t and EQ contains Eqs. (7) and (8).

In contrast, if q_j results in a cache miss, we can get the set of impossible values of $\langle l_j \rangle$, denoted as set $\overline{L_j} = \{\langle l_{m1} \rangle, \langle l_{m2} \rangle, \dots, \langle l_{mr} \rangle\}$. We also introduce variables e_t^k and c_t to describe the comparison between elements in the $\overline{L_j}$ and $\langle l_j \rangle$. The analysis method is the same as the above. As the $\langle l_j \rangle$ is not equal to any $\langle l_{mt} \rangle$ in the $\overline{L_j}$, so that every $c_t = 0$, the algebraic equations can be constructed as below:

$$\prod_{c_t \in C} (c_t \oplus 1) = 1 \quad (10)$$

And the two introduced parameters V and EQ can be calculated as:

$$V = (b+1) \times r, \quad EQ = 1 + (b+1) \times r \quad (11)$$

Solve the System of Equations. After constructing the algebraic equations of both cipher and cache events, we can combine them as the system of equations and then solve it to recover the master key. There are multiple typical methods for solving the algebraic equations, include linearization method [10], Gröbner basis [8] and some SAT solvers like zChaff, Minisat and CryptoMinisat [19]. This paper adopts CryptoMinisat to solve the algebraic equations.

5 The Implementation of ADCA

5.1 Construct Algebraic Equations of SM4 Cipher

As the kernel nonlinear component of cipher, the S-Box is usually the focus of constructing cipher equations. For the SM4, the input size m and output size

Table 1. Descriptions of used variables

Variables	Descriptions
$rk_{i,s}$	The s -th bit in the i -th round key rk_i
$A_{i,s}$	The s -th bit in the input A_i to S-Boxes
$B_{i,s}$	The s -th bit in the output B_i from S-Boxes
$C_{i,s}$	The s -th bit in the output C_i of round function T
$X_{i,s}$	The s -th bit in the intermediate variable X_i

n of S-Box are both 8 bits, i.e., $m = n = 8$. To illustrate better, we first sort out the variables that would be used for constructing equations and list them in Table 1 ($0 \leq i < 32, 0 \leq s < 32$).

Then we can construct the algebraic equations of SM4 algorithm as below:

$$\begin{aligned}
X_{i+1,s} \oplus X_{i+2,s} \oplus X_{i+3,s} \oplus rk_{i,s} \oplus A_{i,s} &= 0 \\
B_i \oplus \tau[A_i] &= 0 \\
C_{i,s} \oplus L(B_i)_s &= 0 \\
C_{i,s} \oplus X_{i,s} \oplus X_{i+4,s} &= 0
\end{aligned} \tag{12}$$

Based on Eq. (12), for the 32-round encryption of the SM4, there are 4352 equations on $GF(2)$, including 1024 nonlinear equations and 3328 linear equations, which contain 5248 variables. As for the key schedule algorithm, with the same analysis method, it derives 4096 equations consisting of 1024 nonlinear ones and 3072 linear ones, with 4224 introduced variables. In summary, there are totally 8448 equations with 9472 variables constructed for the whole process of SM4. However, in the real attack experiments, actually it only needs to construct parts of them since analysis on the first few rounds is enough to reveal the master key.

5.2 Transform Cache Events to Algebraic Equations

To solve above cipher equations, whose solution contains the master key, more constraints are introduced by transforming cache events to algebraic equations. Figure 5 gives three samples of captured cache event sequences, each sequence covers the first 8 rounds (a total of 32 S-Box lookups) of the encryption. The black blocks in the figure denote cache misses while the white blocks denote cache hits. The x -axis indicates the ID of S-Box lookup, y -axis indicates the ID of sample. From the figure, we can know that after the fourth round (i.e., the 16-th S-Box lookup), the frequency of cache hits significantly increases, as almost S-Box lines have been loaded.

We take the sample S_1 in Fig. 5 as an example to explain how to construct algebraic equations for cache events. The initial analysis is similar to the traditional DCA, like the eighth S-Box lookup q_7 in S_1 causes a cache miss, it can get the set of the impossible values of $\langle l_7 \rangle$: $\overline{L_7} = \{\langle l_0 \rangle, \langle l_1 \rangle, \langle l_3 \rangle, \langle l_6 \rangle\}$. Then based

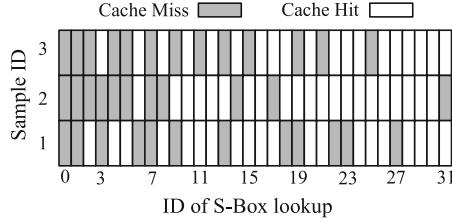


Fig. 5. Samples of cache event sequences.

on the Eq. (11), 20 new variables would be introduced to construct 21 algebraic equations. As for the ninth lookup q_8 , which causes a cache hit, the set of the possible values of $\langle l_8 \rangle$ can be obtained: $L_8 = \{\langle l_0 \rangle, \langle l_1 \rangle, \langle l_3 \rangle, \langle l_6 \rangle, \langle l_7 \rangle\}$. And based on the Eq. (9), there would be 25 introduced variables and 36 constructed algebraic equations.

Now consider the general case of constructing algebraic equations for cache events. Suppose N_c is the number of cache lines that the entire S-Box occupies, N_j is the number of cache lines that have been accessed before the j -th S-Box lookup q_j , and P_j is the probability of that q_j results in a cache hit. Assume N_c cache lines are accessed with equal probability in each S-Box lookup, then for a certain cache line, the probability of it has never been accessed before q_j is $((N_c - 1)/N_c)^j$. After that the values of N_j and P_j can be computed as below:

$$N_j = N_c \times \left(1 - \left(\frac{N_c - 1}{N_c}\right)^j\right), \quad P_j = \frac{N_j}{N_c} = 1 - \left(\frac{N_c - 1}{N_c}\right)^j \quad (13)$$

As the N_j essentially denotes the number of cache misses before q_j (each cache miss would occupy a new cache line), it means that the variable r in the Eqs. (9) and (11) can be substituted with N_j . Hence, for the j -th lookup, the number of introduced variables (V_j) and constructed equations (EQ_j) can be indicated as:

$$\begin{aligned} V_j &= (b + 1) \times N_j \\ EQ_j &= 1 + (b + 1) \times N_j + P_j \times \frac{N_j \times (N_j - 1)}{2} \end{aligned} \quad (14)$$

Following the increase of analysis rounds or namely j , the overheads for both constructing cipher equations and cache event equations become larger. Figure 6 depicts the ratio of the overhead between constructing cache event equations and cipher equations, providing a better view on their variation trend. The blue curve illustrates the overhead ratio of introduced equation of two constructions, while the red curve indicates the overhead ratio of variables. From the figure, the ratio of equation overhead is about 1 in the 8-th round, which means the number of constructed equations for cipher and cache events are nearly equal. After the 8-th round, with the increase of analyzed rounds, the equation overhead increases rapidly while the search space of secret keys reduces slightly. As the attack requires a reasonable compromise between the analysis rounds and the overhead, it seems that analyzing the first 8 rounds is a good choice.

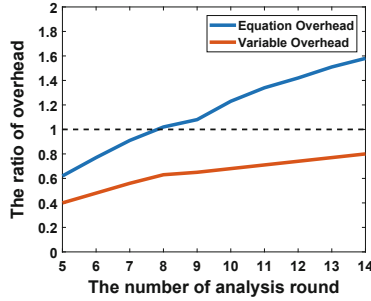


Fig. 6. The overhead of equations and variables. (Color figure online)

5.3 Recover the Master Key

In this paper, we leverage the SAT solver CryptMinisat to solve the system of equations, whose solution can be further used for extracting the master key. The solver CryptMinisat takes CNFs (Conjunctive Normal Forms) as its input, so that above constructed algebraic equations need to be converted to the equivalent CNFs. However, since the CryptMinisat can convert linear equations to CNFs automatically, the analyst actually doesn't need to consider this step. For the cipher equations, each S-Box substitution can be converted to 254 CNFs. After taking the key schedule algorithm into account, each round would generate 1336 CNFs. The cache event equations are also converted to CNFs and sent to the solver together. Then the solver is launched to solve the system of equations automatically [19], which effectively release the analyst from complex analysis. Finally, the master key is extracted from the solved solution.

6 Experimental Evaluation

6.1 Setup

The attack platform for the ADCA experiments is the same to that in Sect. 3, where has $N_c = 16$ and $b = 4$. We assume that side-channel leakages can be captured effectively, and the corresponding cache hit/miss sequences can also be deduced correctly. Then the solving of the system of equations is achieved by the CryptMinisat (version 2.9.4) running on a normal PC (CPU: Intel Core i7-7700 @ 3.60 GHz, Memory: 16G).

6.2 Attack Time

First the time required for a successful attack is studied, i.e., the time of solving the system of equations with CryptMinisat. Note that the attacks exceed 24 h are considered as failed. Based on the depiction in Fig. 6, to effectively reveal the master key without introducing too much overhead, analyzing the first 8 rounds

is an appropriate choice. Figure 7(a) gives the distribution of attack time of 20 experiments, which all analyze the first 8 rounds of SM4 with 10 different plaintext samples. From the figure, we find the attack time almost distributes exponentially, most attacks can be achieved in 12 h (4.32×10^4 s) and nearly 30% of them can be succeeded in less than one hour. The average attack time is about 4.79 h that is affected by the extreme values in the time sequence. Longer attack time is the price of removing complex manual analysis, but as all the computing can be completed by the solver automatically, such time cost is tolerable.

6.3 Influence Factors on Attack Time

After illustrating the approximate distribution of attack time, we further explore the influence factors on attack time. The first is the number of analysis rounds, denoting as R . Empirically, the bigger R would result in shorter attack time, since it offers more constraints to the system of equations. However, it also introduces larger overhead of equations and variables, which makes the target equations more complex. Figure 7(b) shows the variation tendency of attack time with the increase of analysis rounds R , and the sample size of plaintexts is fixed as 20. For each R , 5 experiments are performed to get the corresponding average attack time. The trend indicates that with the fixed number of plaintexts, the increase of R can reduce the attack time, but once R exceeds a certain range (e.g., $R = 7$), the attack time increases rapidly with R . The second parameter is the number of plaintext samples, denoting as S . Similar to the R , the increase of S gives more constraints but also larger overhead. As a result, S has the same influence to the variation of attack time, in a certain range the increasing S makes the attack time decrease, but beyond the range the attack time increases fast. To further illustrate the influence from S to the attack, we discuss the relation between the attack success rate and the plaintext sample size in the following.

6.4 Attack Success Rate

A successful attack means the SAT solver is able to solve the system of equations in a given time (less than 24 h), so the success rate of ADCA is essentially equivalent to the probability of solving the solution in the required time. Figure 7(c) depicts the success rates of the attacks that all take the first 5 rounds for analysis but utilize different size of plaintext samples. The success rate is calculated from 10 repetitive experiments. As the figure shows, the success rate does not always increase with the sample size, when the sample size larger than a certain threshold, the overhead of introduced equations becomes so large that it gets much harder for the solver to solve these equations. Since the solver cannot get the solution in the limited time, it makes the attack success rate decrease rapidly.

Based on above results, in order to conduct a successful algebraic different cache attack, we should choose appropriate analysis rounds R and sample size S , besides we also need to consider the trade-off between these two parameters. In the actual experiment, when the analysis rounds $R = 8$ and the sample size

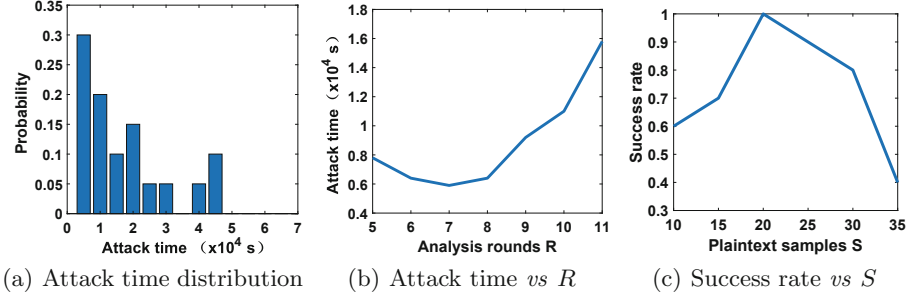


Fig. 7. Results of experiment evaluation.

$S = 10$, the attack can obtain the best result, whose attack time is short and success rate is high.

7 Error-Tolerant Enhancement

In above attack experiments, it assumes that all the deduced cache events are correct, so that we can directly construct algebraic equations of them to solve the key. However, in practice, the cache events deduced from side-channel leakage traces are always contain more or less misjudgments. There are two main reasons: the noise that fluctuates the accurate leakage traces and the improper threshold that mislead the inference. As such error cache events would compromise the analysis or even make the attack failed, it is necessary to propose an error-tolerant method to improve the robustness and practicality of ADCA.

7.1 Error-Tolerant Method

Since errors in cache events are essentially misjudgments of captured side-channel leakages, our error-tolerant method focuses on the interpretation of leakage traces. In general, the noise of instruments and environment is the white noise, which changes the values of monitored leakages in a certain range. To deal with such attached noise, a flexible threshold range is adopted to decide the type of cache events, i.e., cache hit or miss. It can efficiently filter most of the noise and make sure the correctness of deduced events. Besides that, since the judgment of cache events depends on a range rather than an accurate value, it also reduces the sophisticated works for choosing a strict threshold.

The error-tolerant method is designed to enhance the ADCA. First, two thresholds V_M and V_H are set, the former is the lowest peak value that can be deduced as cache miss, and the latter is the highest peak value that can be seem as cache hit. Let V_j denote the peak value in the leakage trace that corresponds to the j -th S-Box lookup q_j . After that, the cache events can be judged from the captured traces as following:

1. if $V_j \leq V_H$, q_j is considered as a cache hit, then constructing equations of cache events based on the Eqs. (7) and (8).
2. if $V_j \geq V_M$, q_j is considered as a cache miss, then constructing equations of cache events based on the Eqs. (7) and (10).
3. if $V_H \leq V_j \leq V_M$, it is difficult to confirm whether q_j is a cache hit or cache miss, so mark it as the uncertain event and leave it alone.

For the cache events whose V_j satisfies $V_H \leq V_j \leq V_M$, as they are ambiguous, the best solution is ignoring them. Although dropping them would result in the information loss of analysis, it is still better than introducing errors. And the choice of V_M and V_H must be careful, it should balance between the information loss and introduced errors. In general, first a number of experiments should be performed on the current platform to determine the base line between cache hit and cache miss. Then, combining the results in the captured leakage traces, the values of V_M and V_H are chosen by shifting up or down the base line.

In order to better explain the proposed error-tolerant method, the uncertainty ratio e is introduced to describes the proportion of uncertain cache events in the population. The parameter e can be denoted as: $e = \frac{N_E}{N_A}$, where N_E is the number of uncertain cache events, and N_A is the number of all cache events. The higher e generally means the larger noise in the experiments and the harder it is to obtain the correct key. Besides, it should be noted that the choice of V_M and V_H also make a great influence to the value of e .

7.2 Evaluation

The setup for evaluating the error-tolerant attack is the same as that shown in Sect. 6, and the attack analyzes the first 8 rounds of encryption. As we cannot manipulate the noise of instruments and environment as will, the values of V_M and V_H are adjusted to achieve changing the uncertainty rate e , which has the same effect as the variation of noise. Then the efficiency of attacks under different e is evaluated. Figure 8 depicts the average sample size of plaintexts required for revealing the key under different uncertainty ratio e . The curve in the figure indicates that the required sample size increases exponentially with the e , and such

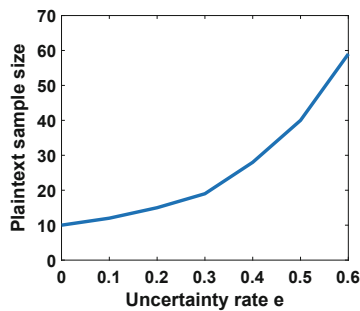


Fig. 8. Required plaintext sample size for attacks with different error ratio e

observation is in line with our expectations. Our error-tolerant attack method is able to reveal the key with 59 plaintext samples when the uncertainty rate e is 60%, and while e is less than 30% the method can recover the master key of SM4 within 20 input samples.

8 Conclusions

This paper enhances the traditional differential cache attack with algebraic analysis and error-tolerance, proposing an error-tolerant algebraic differential cache attack method. With applying on the block cipher SM4, it shows that the new attack makes a great reduction in the attack complexity and bypass the sophisticated manual analysis effectively. Only 10 plaintexts with the analysis on the first 8 rounds are enough to recover the master key of SM4, and the analysis is automatically finished by the SAT solver. The proposed error-tolerant method further improves the robustness and practicality of the attack, which allows to reveal the cipher key successfully when the uncertainty rate is less than 60%. Such work provides the reference for following physical security evaluation on other typical block ciphers, such as ARIA and Camellia.

Acknowledgments. This work was supported in part by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, by Zhejiang Key R&D Plan (Grant No. 2019C03133), by Major Scientific Research Project of Zhejiang Lab (Grant No. 2018FD0ZX01), by Open Fund of State Key Laboratory of Cryptology (Grant No. MMKFKT201805), by National Natural Science Foundation of China (Grant No. 61772236), by Young Elite Scientists Sponsorship Program by CAST (Grant No. 17-JCJQ-QT-045), by National Natural Science Foundation of China (Grants No. 61772236, 61972348) and by Research Institute of Cyberspace Governance in Zhejiang University.

References

1. OpenSSL 1.1.1 source code. <https://www.openssl.org/source/old/1.1.1/>
2. The SM4 block cipher algorithm and its modes of operations (2019). <https://tools.ietf.org/id/draft-crypto-sm4-00.html>
3. Acıgmez, O., Schindler, W., Koç, Ç.K.: Cache based remote timing attack on the AES. In: Cryptographers' Track at the RSA Conference, pp. 271–286 (2007)
4. Bai, X., Guo, L., Xu, Y., Li, Z.: Research on differential power analysis attack on SMS4 algorithm. *J. Chin. Comput. Syst.* **3**, 541–544 (2009)
5. Bertoni, G., Zaccaria, V., Breveglieri, L., Monchiero, M., Palermo, G.: AES power attack based on induced cache miss and countermeasure. In: International Conference on Information Technology: Coding and Computing (ITCC 2005)-Volume II, vol. 1, pp. 586–591 (2005)
6. Chen, J., Hu, Y., Zhang, Y.: Impossible differential attack on the 17-round block cipher SMS4. *J. Xidian Univ.* **3**, 455–488 (2008)
7. Cho, J.Y., Nyberg, K.: Improved linear cryptanalysis of SMS4 block cipher. In: Symmetric Key Encryption Workshop, pp. 1–14 (2011)

8. Erickson, J., Ding, J., Christensen, C.: Algebraic cryptanalysis of SMS4: Gröbner basis attack and sat attack compared. In: Lee, D., Hong, S. (eds.) ICISC 2009. LNCS, vol. 5984, pp. 73–86. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-14423-3_6
9. Gallais, J.F., Kizhvatov, I.: Error-tolerance in trace-driven cache collision attacks. In: International Workshop on Constructive Side-Channel Analysis and Secure Design, COSADE, pp. 222–232 (2011)
10. Ji, W., Hu, L., Ou, H.: Algebraic attack to SMS4 and the comparison with AES. In: 2009 Fifth International Conference on Information Assurance and Security, vol. 1, pp. 662–665 (2009)
11. Lou, X., Zhang, F., Huang, J., Zhao, X., Liu, H.: Research on trace driven cache analysis on SM4. *J. Cryptologic Res.* **5**(4), 430–441 (2018)
12. Neve, M., Seifert, J.P.: Advances on access-driven cache attacks on AES. In: Biham, E., Youssef, A.M. (eds.) *Selected Areas in Cryptography, SAC 2006. Lecture Notes in Computer Science*, vol. 4356, pp. 147–162. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-74462-7_11
13. Nguyen, P.H., Rebeiro, C., Mukhopadhyay, D., Wang, H.: Improved differential cache attacks on SMS4. In: Kutyłowski, M., Yung, M. (eds.) *Inscrypt 2012. LNCS*, vol. 7763, pp. 29–45. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38519-3_3
14. Osvik, D.A., Shamir, A., Tromer, E.: Cache attacks and countermeasures: the case of AES. In: Pointcheval, D. (ed.) *CT-RSA 2006. LNCS*, vol. 3860, pp. 1–20. Springer, Heidelberg (2006). https://doi.org/10.1007/11605805_1
15. Page, D.: Theoretical use of cache memory as a cryptanalytic side-channel. *IACR Cryptology ePrint Archive* 2002/169 (2002)
16. Poddar, R., Datta, A., Rebeiro, C.: A cache trace attack on CAMELLIA. In: Joye, M., Mukhopadhyay, D., Tunstall, M. (eds.) *Security Aspects in Information Technology. InfoSecHiComNet 2011. Lecture Notes in Computer Science*, vol. 7011, pp. 144–156. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-24586-2_13
17. Rebeiro, C., Mukhopadhyay, D.: Cryptanalysis of CLEFIA using differential methods with cache trace patterns. In: Kiayias, A. (ed.) *CT-RSA 2011. LNCS*, vol. 6558, pp. 89–103. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19074-2_7
18. Rebeiro, C., Poddar, R., Datta, A., Mukhopadhyay, D.: An enhanced differential cache attack on CLEFIA for large cache lines. In: Bernstein, D.J., Chatterjee, S. (eds.) *INDOCRYPT 2011. LNCS*, vol. 7107, pp. 58–75. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25578-6_6
19. Soos, M., Nohl, K., Castelluccia, C.: Extending SAT solvers to cryptographic problems. In: Kullmann, O. (ed.) *SAT 2009. LNCS*, vol. 5584, pp. 244–257. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02777-2_24
20. Tsunoo, Y., Saito, T., Suzaki, T., Shigeri, M., Miyauchi, H.: Cryptanalysis of DES implemented on computers with cache. In: Walter, C.D., Koc, Ç.K., Paar, C. (eds.) *CHES 2003. LNCS*, vol. 2779, pp. 62–76. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45238-6_6
21. Zhang, M., Liu, J., Wang, X.: Differential attack on 22-round SMS4 block cipher. *Acta Scientiarum Naturalium Univ. Sunyatseni* **2**, 43–47 (2010)
22. Zhao, X., Wang, T., Zheng, Y.: Cache timing attack on SMS4. *J. Commun.* **6**, 89–98 (2010)
23. Zhong, M., Hu, Y., Chen, J.: Square attack on the 14-round block cipher SMS4. *J. Xidian Univ.* **1**, 105–109 (2008)